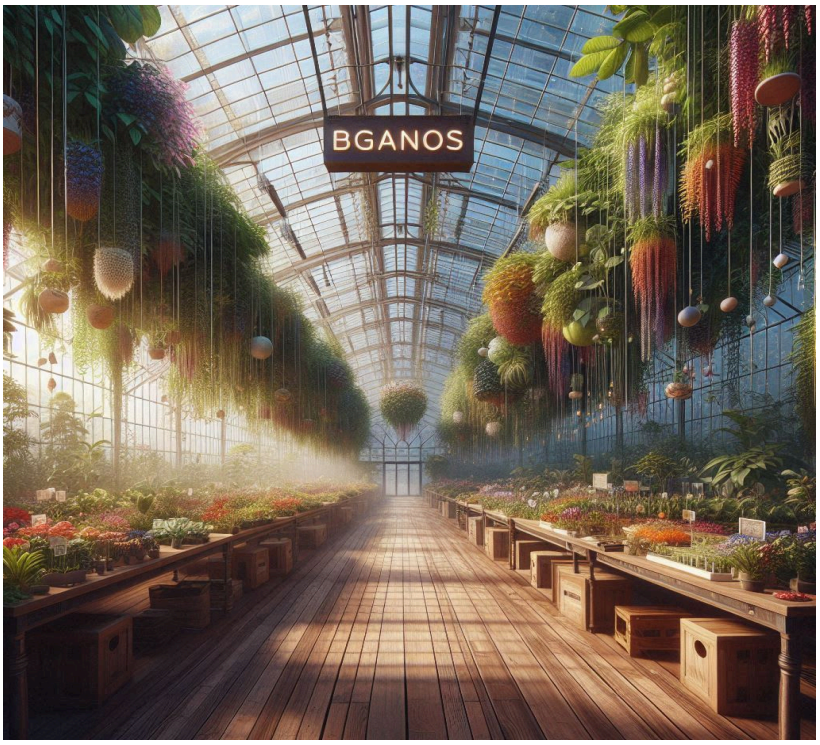
Memoria del proyecto

BGANOS



Modelado del Software

Grupo E

2024-25

Integrantes:

Iván Alcalde Cámara

Ángel Antequera Gómez

Alicia Chen

Melany Daniela Chicaiza Quezada

Pablo Cruz Sojo

Lucía García García

Airam Martín Peraza

Ignacio Melendo Bruggeman

Diego Montero Blanco

Luis Rebollo Cocera

Sergio Vázquez Carbajo

Jiayun Zhan

1. Introducción

Mediante el siguiente documento se pretende dejar reflejado los detalles técnicos y algunos detalles del equipo, como la metodología de trabajo, seguidos durante la realización del proyecto. Los puntos presentados son la arquitectura, los patrones usados, la metodología de trabajo seguida, las herramientas utilizadas y los repositorios donde se encuentra todo lo relacionado con el proyecto. Al final se indica también donde se puede encontrar la nota para cada integrante del grupo según el trabajo realizado de cada uno.

2. Arquitectura

Para la implementación de este proyecto se ha usado una arquitectura multicapa, dividida en 3 capas principales: Presentación, Negocio e Integración. También se ha dividido por columnas, es decir, se ha creado un elemento por módulo para cada elemento de cada capa.

La combinación de ambas divisiones nos da una aplicación dividida horizontalmente por capas, pero también dividida verticalmente por módulos.

Las principales ventajas de usar la arquitectura multicapa son, entre otras, el bajo acoplamiento, es decir, que módulos de diferentes capas se pueden cambiar, sin que interfieran en las otras capas, esto nos permite realizar cambios en una capa sin estar obligados a cambiar todas las capas. También nos permite poder trabajar en paralelo en las diferentes capas.

2.1 Capa de Presentación

En la capa de presentación se encapsula toda la lógica de vistas necesaria para dar los servicios a los usuarios que usan el sistema. Las vistas son diferentes para los datos de entrada y los de salida, es decir, para cada vista hay mínimo otra que muestra la información de salida correspondiente al servicio solicitado. Las vistas se implementan mediante la herramienta que ofrece Java, Swing. Cada una de las vistas implementa una interfaz general de vistas, que contiene un método llamado “actualizar”, para poder tratar a todas las vistas bajo el mismo tipo. El proceso de trabajo de las interfaces de usuario es muy sencillo, cuando el usuario introduce los campos necesarios para el servicio requerido, mediante un listener, el evento asociado entra en acción, llamando al Controlador, el cual es el encargado de comunicar las vistas con los comandos. Los comandos transmiten la información encapsulada en un Transfer al negocio, tras recibir la respuesta, genera un evento correspondiente a la salida, el cual es transferido al controlador, quien genera una vista mediante una factoría, y la actualiza.

2.2 Capa de Negocio

En la capa de negocio se encuentra toda la lógica necesaria para que el sistema responda de acuerdo con los requisitos especificados en la SRS. Esta capa cuenta con un servicio de aplicación por módulo, es decir, 6 en total, los cuales contienen todas las operaciones correspondientes a cada módulo. Los servicios trabajan con objetos de negocio, los cuales contienen anotaciones correspondientes a JPA. Cuando un comando ejecuta una 4 operación del servicio, este procesa la información recibida, y, si tiene que grabar o leer datos de la base de datos, envía una petición a la misma. Tras realizar el procesamiento de los datos, genera una respuesta que devuelve al comando.

2.2 Capa de Integración

La capa de integración no está implementada por nosotros, esta capa existe de manera implícita. Está implementada por JPA, y es el encargado también de gestionar las peticiones dirigidas a la base de datos con sus propios métodos. La conexión a la base de datos también es gestionada por JPA, pero esta debe ser configurada por los implementadores de la aplicación.

3. Patrones

En el siguiente punto se especifican los patrones utilizados para la implementación de la aplicación.

- **Patrón Transfer**: simplifica la transferencia de datos entre capas.
- **Patrón SA** (servicio de aplicación): centraliza toda la lógica de negocio.
- **Patrón Singleton**: garantiza el acceso único a una clase mediante una misma instancia, controlando que haya siempre un número controlado de instancias.
- **Patrón Factoría abstracta**: permite crear un conjunto de clases dependientes, mediante una única interfaz, abstrayendo la creación a una sola clase.
- **Patrón Service to worker**: de este patrón aplicamos los siguientes
    - El controlador de aplicación, el cual centraliza la gestión de llamadas a vistas o a negocio en una clase.
    - El patrón command, que sirve como intermediario entre el controlador de aplicación y los SAs, ya que cada comando ejecuta un caso de uso, desacoplando así la llamada a negocio del controlador

  El patrón de **view** helper no lo aplicamos ya que las vistas no son lo suficientemente complejas como para necesitarlo, en caso de aplicarlo en este caso, complicaría más la depuración del código añadiendo llamadas y otras clases.

  El patrón de **controlador frontal** tampoco se aplica al no gestionar un servicio web.

- **Objeto Negocio**: se asemejan de cierta manera a los transfers pero los objetos de negocio permiten tener punteros a otros objetos e implementar cierta lógica de negocio.
- **Command**: este patrón encapsula los eventos y los datos recibidos de las vistas, además permite añadir más funcionalidades a la aplicación, sin tener que cambiar mucho el resto de la aplicación, más allá de unos cuantos detalles.

4. Gestión del proyecto

Para organizar el trabajo a realizar de la segunda entrega del proyecto BGanos se optó por un modelo de trabajo utilizando la herramienta de GitHub projects. Este nos provee de un tablero de trabajo (KanBan) en el que podemos ir actualizando y visualizando el flujo de trabajo además de los deadlines que fuimos estableciendo para intentar llevar el proyecto al día.

5. Recursos utilizados

Para la realización de este proyecto se han requerido de diferentes herramientas, a continuación, se detallan todas las utilizadas y para que se han utilizado cada una. o IBM RSAD: para la realización de todo el modelo, desde el diagrama de clases, hasta los diagramas de secuencia, componentes y el modelo del dominio. o Eclipse IDE: para la implementación del código Java y los JUnits. o MySQL Server y Workbench: para la gestión integral de la base de datos. o GitHub: como sistema de control de versiones.

6. Repositorios

La información se encuentra distribuida en 2 repositorios diferentes de código y diseño.

- Diseño: https://github.com/anavarro-fdi-ucm-es/ms2425bganosmod
- Código: https://github.com/anavarro-fdi-ucm-es/ms2425bganoscod
- Documentación: se encuentra en la wiki del repositorio de diseño.

7. Ponderaciones

Por último, se proporciona una tabla con el trabajo realizado por cada integrante según el total, de 0.75 a 1.25.

Iván Alcalde Cámara	1
Ángel Antequera Gómez	1
Alicia Chen	1,10
Melany Daniela Chicaiza Quezada	1
Pablo Cruz Sojo	1
Lucía García García	1
Airam Martín Peraza	1
Ignacio Melendo Bruggeman	1
Diego Montero Blanco	1
Luis Rebollo Cocera	0,98
Sergio Vázquez Carbajo	1
Jiayun Zhan	1